

ECOCIDE: A Causal Inference Framework for Independently Verifying War Time Environmental Damage

Note: entries are a chronological diary and describe what I believed at the time. Early entries (and this introduction) use stronger language — “prove”, “conflict attributable” — than the final results support. Current results are in `ECO_RESULTS_RECONCILIATION.md` and Entry 17.

So...I keep detailed notes on almost every research project I take on..... It began as a purely practical habit—something to stop me from waking up the next morning completely lost about where I’d stopped or which exact files and datasets were sitting on my machine.

With this particular project though, I decided to turn those notes into a continuous narrative..... So I wanted to capture the real sequence of decisions, the abrupt shifts in data sources, the frustrating bugs that brought everything to a halt, and the long debugging stretches that eventually fixed them.....The goal was simple: leave behind an honest, unfiltered record of how the work actually unfolded, so anyone curious can follow the full path from start to finish.

So I built ECOCIDE as a geospatial causal inference framework to cross check and prove war related environmental damage on my own. Instead of just staring at before and after satellite pictures and making wild guesses, I wanted to use actual Earth Observation data combined with solid statistical math. Right now, international courts are looking at “ecocide”—which basically means destroying nature on a massive scale—as a serious war crime you can actually prosecute.

But if you look closely at the current maps and reports feeding this global debate, even the newest papers on the Ukraine war completely refuse to prove real statistical causality. They keep playing it safe by using words like “conflict associated” instead of flat out calling the damage “conflict attributable.” (They just note that a change happened, not that the war definitely caused it.)

My ECOCIDE framework fixes this exact missing puzzle piece. I used a strict Difference in Differences model and verified the whole math setup using fake placebo trials and event study graphs. To test if it actually works, I applied this entire code to the Kakhovka Dam disaster, benchmarking the damage against a separate, matching control zone that never saw any combat.....

Index

1. [Entry 1](#)
 2. [Entry 2](#)
 3. [Entry 3](#)
 4. [Entry 4](#)
 5. [Entry 5](#)
 6. [Entry 6](#)
 7. [Entry 7](#)
 8. [Entry 8](#)
 9. [Entry 9](#)
 10. [Entry 10](#)
 11. [Entry 11](#)
 12. [Entry 12](#)
 13. [Entry 13](#)
 14. [Entry 14](#)
 15. [Entry 15](#)
 16. [Entry 16](#)
 17. [Entry 17](#)
 18. [Entry 18](#)
-

Entry 1

Began by locking down the exact research question. Didn't want to just jump straight into downloading data files without a plan. Look at all the current published geospatial papers on Ukraine's war damage. They basically just do visual, qualitative before and after satellite image checks. That's it. They outright refuse to prove any statistical causality. They keep calling the environmental changes "conflict associated" instead of "conflict attributable." (Honestly, that is fair enough since their only goal was a broad description.)

But this leaves a massive, known gap in the research. I searched everywhere but couldn't find a single published framework that actually measures conflict attributable environmental damage with a real causal inference setup. To fix this, I needed a proper matched non conflict control zone, a Difference in Differences model, and placebo validation.

So, that specific gap became my real goal. Set out to build a reproducible geospatial framework that actually calculates environmental damage tied directly to armed conflict. It had to separate conflict driven changes from the normal baseline environmental trends, giving me proper statistical confidence instead of just another basic, qualitative before and after image matchup. (No more lazy comparisons for me.)

Chose the Kakhovka Dam destruction from June 6, 2023, along with the nearby Dnipro River floodplain and the Donbas industrial region, as my main validation case. Why this specific event? Because the whole conflict timeline is super well documented, and the pre conflict environmental baselines are easily available to test my model properly.....

Entry 2

So I had to fix my treatment and control zones before touching any actual data files..... Setting up the treatment zone was very easy. So I picked the Kakhovka Dam area and the downstream Dnipro River floodplain (46.777°N, 33.370°E) inside Kherson Oblast, Ukraine. UNOSAT's own satellite data showed that this flood affected analysis zone covers about 10,800 km² tracking all the way from the dam down to the river mouth.

Finding a good control zone was much harder. At first, I thought about picking a peaceful area inside Ukraine like Dnipropetrovsk Oblast, but I dropped that idea fast. News reports showed that the frontlines were moving closer to that area, meaning economic issues, supply chain breaks, and refugees moving around could mess up my control data even if no bombs were dropping right there. (I actually made this exact mistake before in my GPIE project, where using a control group from inside the same war region failed because the areas were too connected.)

So, I chose the Danube Delta in Tulcea County, Romania (45.200°N, 29.500°E) instead. It gives me a perfect, matching pre conflict ecological baseline because it has the same river delta wetlands, Pannonian steppe, farming floodplains, and a similar continental climate, but it is 100% safe and non combatant..... I manually verified that this is completely separate from the Ukrainian side of the Danube Delta in Odesa Oblast, which is a good thing because Odesa has been hit by war strikes on its port infrastructure and would completely ruin my control baseline.

Entry 3

So I had to pull administrative boundaries for both zones next. So I used GADM version 4.1 for this, grabbing the massive, full country level GeoPackage files for Ukraine and Romania. (GADM does not let you just download a single region on its own, which is a bit annoying.)

When I started extracting the shapes, I hit a stupid naming bug..... For Ukraine, the oblasts sit cleanly at GADM Level 1 under the column NAME_1, so grabbing Kherson was super easy. But Romania's file threw a total wrench in my script. Level 2 in Romania's schema doesn't show counties at all—it actually shows tiny local communes and small municipalities with hundreds of random place names. Because of this layout mismatch, Tulcea just wasn't there.

I jumped back to check Level 1, and yep, that is where Romanian counties actually live..... So Tulcea finally extracted perfectly from that tier. This wasn't a case of corrupted files or bad data; it was just a genuine, weird inconsistency in how GADM numbers its administrative hierarchies across different countries. It taught me a good lesson: always inspect the actual level structure of each country first instead of blindly assuming the database numbering stays the same everywhere.

Anyway, I saved out both boundaries into separate files. The final verified bounding boxes came out to: Kherson at 31.51°E-35.10°E, 45.90°N-47.58°N, and Tulcea at 27.99°E-29.72°E, 44.61°N-45.46°N. :)

Entry 4

Next, I pulled the monthly NDVI data for both bounding boxes. Used the Sentinel Hub Statistical API to get data from January 2022 right through to November 2024. This gave me exactly 35 monthly data points for each zone. To do this quickly, I just reused the same old login module and evalscript setup that I had already coded for my earlier GPIE and DOUBLE JEOPARDY projects..... (Saved me a lot of extra coding time.)

So yeah, the raw values finally arrived. They came back looking very normal, sitting between 0.07 and 0.17 for the cold winter months. This makes perfect sense for a steppe and farming area because the plants go to sleep during winter. Since the numbers looked solid and matched seasonal vegetation dormancy expectations, I chose to accept them as correct and didn't add any extra data cleaning steps at this point.

Entry 5

Man, NDWI tracking became a total headache with so many bugs. My very first data pull used the entire Kherson Oblast bounding box at a basic monthly resolution, but it literally showed absolutely nothing around the June 6, 2023 dam blast. So I refused to just accept this zero result and quit, so I started looking deeper into the code. The math problem was clear: Kherson Oblast is huge at about 28,000 km², but UNOSAT records showed the actual water leak covered only around 600 km². So, when my script averaged the NDWI across the whole massive oblast box, the actual flood signal—which was stuck in just 2% of the space—got completely drowned out by the other 98% of dry land.

So I changed tactics. Cropped the bounding box much tighter around the river corridor and the floodplain. This helped a tiny bit, showing a slightly better reservoir filling pattern in May 2023, but June and July still had no clean peak..... Then, two whole months came back with completely blank data. (This happens because Sentinel-2 is an optical satellite, meaning it is blind when winter clouds cover the sky.)

Changed the code to weekly resolution on that same tight river corridor, but things went completely downhill, though it taught me something useful. On June 3, 2023—literally the exact week before the dam collapsed—the data gave me the lowest, most negative, least water-like number in the entire time series. It was crazy because it showed the exact opposite of what a massive flood should look like.....

I finally tracked this down to a big structural flaw in my coordinate box..... The issue was that the box covered both the upstream Kakhovka Reservoir—which dried

up super fast losing around 18 km³ of water—and the downstream floodplain area tht got completely soaked at the exact same time. These two sub regions were moving in completely opposite directions so the math just cancelled out nd left me with total noise. (Wht a mess.)

I tried splitting the box right at the dam's latitude line to make separate top and bottom zones, but that did not fix the issue. My data still showed crazy, impossible jumps from week to week. For example, the water percentage in one area leaped from 45% to 16% nd then dropped below 3% in three straight weeks without any real physical explanation.

This weird bouncing was just sensor noise, not real water movement on the ground. Sentinel-2 gets blocked by clouds all the time, meaning the code takes stats from whatever random cloud free pixels are open that week so the baseline shifts every single time. The other published papers on Ukraine's ecocide avoid this exact mess by using radar based flood tracking maps instead of tricky optical ones.

So I switched my code for the upstream zone to Sentinel-1 SAR. (Thank god radar can easily pierce right through heavy cloud cover.) I applied a standard -17 dB threshold on the VV polarization band to separate out the water pixels. This tweak worked beautifully nd gave me a very realistic, clean signal. :) The math showed water levels holding steady around 2% to 5% until late May 2023, but then it crashed heavily during the June 3 week. After tht, the water stayed super low between 1% nd 3% all the way through August without bouncing back up. This matches the actual event perfectly because the reservoir emptied out and never refilled.

But, the exact same SAR setup completely failed to give a clean reading for the downstream floodplain area. The values jst kept jumping up nd down within a tiny 4% to 6% range, nd I even caught a weird, random dip right after the dam broke. So I think this glitch happened because the entire high flood nd drying cycle took place inside a single weekly data window, or maybe because muddy, wet farm fields create a weird radar reflection tht completely misses the baseline threshold I set for deep reservoir water. In the end, I decided the top upstream signal was clean enough to start coding my actual causal model, so I jst left the messy downstream data as an unfinished problem instead of forcing the same script to fit where it shouldn't.

Looking back, this whole process was quite a journey. Every time I hit a weird result or a missing data signal, I didn't just give up or cherry pick whatever number looked nice. Instead, I treated every single issue as a debugging puzzle to solve. (I had to deal with spatial dilution, opposite water movements cancelling each other out inside one single box, and clouds blocking my optical views.)

Every single fix I made came from finding a real, proven physical reason in the geography, not just blindly turning random knobs or tweaking parameters until the code worked. For tht one downstream floodplain part tht I jst couldnt fix, I chose to be completely honest nd wrote it down as an unsolved problem in my final report.

Entry 6

Okayyy so, I stopped wasting time tweaking thresholds on raw satellite bands for the downstream floodplain. Instead, I just changed my plan n looked for a ready made, verified flood map from an official group. (Other papers on Ukraine ecocide did the exact same thing for this event anyway.) I searched online n found a really detailed UNOSAT flood dataset on the Humanitarian Data Exchange portal with the code FL20230606UKR. It was great because it used data from lots of different satellites like ICEYE radar, Landsat-9, SkySat, WorldView-3, and MODIS Aqua/Terra across multiple dates from June 3 to June 21, 2023, and UNOSAT checked everything for quality control.

So I loaded the June 6, 2023 file into my script. It was a single polygon map in the

EPSC:4326 system, showing abt 122.50 km² of water on tht day. This number was smaller than the final cumulative figure of around 600 km² tht UNOSAT reported later. But tht makes perfect sense because a flood grows slowly over two weeks rather than filling up everything on the very first day.

Nxt, I loaded all five available day maps from June 6, 8, 9, 13, nd 21. The final hydrograph graph looked very realistic: the water grew fast to a peak of 464.18 km² on June 9, nd then went down slowly to 21.17 km² by June 21. The whole up and down cycle finished in jst two weeks..... This finally explained why my own weekly NDWI n SAR code nvr caught a clean signal..... The actual flood was moving way too fast so the regular satellites couldn't take clear pictures on time when there were no clouds. Using this verified multi sensor map fixed my downstream measurement issue completely, saving me from endless parameter tuning that was nvr going to work anyway.

Entry 7

Then I built the actual causal model..... It took me a few tries to get something I could actually trust. My first Difference in Differences run compared the monthly NDVI between Kherson (the treatment area) and Tulcea (my control group) across the whole 2022 to 2024 timeframe using June 2023 as the start of the treatment. The math came back looking totally weak nd non significant with an R² of 0.054 nd a p value of 0.117. (Honestly, I didn't just throw my hands up nd call it a failed test.) Instead, I used that low R² score as a hint to find the problem because NDVI has huge, well known seasonal cycles so a model tht only explains 5% of the variance means I forgot a major control variable, not tht there is no real effect.

So I added month fixed effects to the regression. Wow, my guess was totally right. The R² score jumped straight up to 0.747, and the exact same coefficient value for did_term came out to -0.0703, which is now statistically significant with a p value of 0.007. The actual impact number did not shift at all; only the calculated precision got better. This makes complete sense because winter n summer noise was inflating the uncertainty around a real effect, so fixing that seasonal issue proved the link wasn't just some random error from a missing variable.

To double check everything, I ran a placebo test using a fake treatment date of June 2022, looking only at data from before the dam actually broke in June 2023. It came back almost at zero n completely non significant with a score of 0.0148 and a p value of 0.741. A good placebo check needs a tiny coefficient n a high p value at the same time, nd since my code hit both targets, it is basically the strongest proof you can get without running a real randomized lab experiment. Yay.

Tried to push my luck with a full monthly resolution event study next. So I wanted to run the exact same type of quality check that worked so well in my GPIE 23 quarter model, but this time the code completely crashed out on a technical level. The data was just too tiny. With only 70 total observations trying to estimate abt 68 parameters, my regression went rank deficient and spit out NaN errors for every single p value. (Wht a frustrating bug.) This over parameterization mess happened because this specific project has a super small dataset compared to the massive 27 country panel I used in GPIE, so the math itself isn't broken, the data size is jst too low..... To fix this, I dropped the fine monthly track n grouped the data into quarterly bins instead which successfully brought the parameter count way below the observation count.

But the new quarterly event study gave me a big headache instead of a clean win. The post event quarters looked fine n showed real negative impacts, like Quarter 0 having a p value of 0.035 nd Quarter +4 hitting a p value of 0.0001. But then I noticed a weird anomaly: one pre event quarter—summer 2022—also came back statistically significant with a p value of 0.007. This completely breaks the clean parallel trends assumption you need for this model. So I didnt jst ignore it; I dug into the history files to find out why. Turns out, Kherson Oblast was already a

massive warzone long before the dam broke, especially with the heavy Kherson liberation fighting happening from August to November 2022. So my original pre period wasn't a clean, peaceful baseline at all; it was just a phase of a different, ongoing fight. This is a big scope problem because I am trying to measure just the dam's damage, not the total build up of the whole war.

So I chopped the pre period down to just January to May 2023. This window sits right before the dam broke..... I re-ran the code. This time, I got a much bigger and highly significant drop with a did_term of -0.1384, a p-value of 0.002, and an R^2 of 0.768. This is exactly what you expect when you narrow down the dates properly to isolate the specific event damage..... But then I ran a placebo check inside this short window using a fake date of March 2023. The final coefficient number came out to -0.1382. It was almost identical to my real result but it wasn't statistically significant since the p-value was 0.169. (Honestly, this is a much weaker validation because a good placebo needs to be super close to zero.) The window only had 10 total observations so the statistical power was incredibly low..... I couldn't tell if this was a true zero or just a weak test on a messed up data trend, so I just wrote it down as an open limitation instead of cheating or hiding it.

So I chose to keep both model runs in my final report instead of just picking the one that looked better. Put the broad baseline result (-0.0703, $p=0.007$) as my main finding because its placebo check was 100% clean and correct. Then, I added the tight baseline model (-0.1384, $p=0.002$) as a side sensitivity check, clearly showing its weird placebo limitation. This whole process follows the exact same working style I used back in my GPIE and DOUBLE JEOPARDY assignments..... You take a good looking number, treat it like a guess that needs heavy stress testing, find out the real cause of code failures, and report any messy data as messy.....

Entry 8

So I tried to calculate the reservoir and floodplain water loss using the exact same Difference in Differences setup as my NDVI model, but it was just not possible. (There is literally no matching control zone anywhere on earth for a massive river mouth reservoir collapse.) Tulcea's Danube Delta does not have any huge upstream dam setup like that.

Instead of forcing a fake comparison that makes no sense, I just wrote down the water changes using simple descriptions. I reused the verified UNOSAT flood maps for this..... The records showed the reservoir size before the break was around 2,155 km^2 holding 18.2 km^3 of water. I matched this against the downstream flood which peaked at 464.18 km^2 on June 9 and then shrank back down to 21.17 km^2 by June 21. So I put these raw numbers in just to show the massive physical scale of the disaster next to my main NDVI graph, not as a separate causal test.....

Entry 9

So yeah, once the causal model, event study, and flood analysis were done, I turned the validated numbers into the paper's actual figures and maps, built a Streamlit dashboard to present everything interactively, and put the whole project up on GitHub.

Entry 10

Once the project was mostly finished, I went back through the statistics one more time to make sure they would hold up under real scrutiny, and the standard error setup turned out to be a genuine problem.

At first, my models just used basic OLS standard errors everywhere which is a bad choice for this layout. Since I am only tracking two geographic areas over time,

cluster robust errors—the exact tool I used for my past multi country panels—completely break down because cluster math requires lots of different groups, not just two.

So I switched to Newey West HAC standard errors instead because they are the right fix when you have a small number of long time series, and I updated all five of my causal inference code files. My main result and the tight baseline calculation both survived this change. In fact, the tight baseline number actually became even more significant.....

But there was one major shift: the tight baseline placebo test, which used to look confusing and uncertain under regular OLS errors, came back highly significant under the new HAC calculations. This means it fails completely as a validation test now. I didn't try to hide this with soft words; I just wrote it down as a clear validation failure in both my paper and on the dashboard. It does not ruin my main conclusion anyway because the primary broad baseline model's placebo check stays perfectly clean under the exact same HAC formula.

Entry 11

I went back through every single math claim in my research paper next. Completely re-calculated everything using the raw data files directly, rather than trusting the old numbers that were lying around on my desktop from past work sessions. So I re-derived the whole pre-treatment covariate balance check completely from scratch. The numbers came out to: Kherson $n=17$, $\text{mean}=0.222$, $\text{SD}=0.074$, and Tulcea $n=17$, $\text{mean}=0.203$, $\text{SD}=0.110$, giving a t-test p-value of 0.553. Then, I opened up the raw UNOSAT flood shapefiles again and changed their projection to EPSG:6933 to calculate the area sizes. The sizes came out to exactly 122.50 km², 464.18 km², and 21.17 km² which was a perfect match. Lastly, I re-ran all five of my causal inference Python scripts directly against the raw NDVI files to be absolutely sure.

Everything matched up perfectly. Wohoo. The main DiD script reproduced a `did_term` value of -0.0703, showing an HAC p-value of 0.022 and a basic OLS p-value of 0.0070. The broad baseline placebo run gave me 0.0148 with an HAC p-value of 0.6124. My tight baseline script reproduced a drop of -0.1384 with an HAC p-value of 0.000129, and its placebo check came out to -0.1382 with an HAC p-value of 0.0011. (This clearly caught that weird 'not significant under classical but highly significant under HAC' error pattern which proves the validation check failed.) All four quarters in my event study graph matched up to the exact decimal point, even showing that early pre-treatment glitch I found before.....

The only number I just couldn't re-calculate on my own was the ~10,800 km² downstream analysis corridor estimate used to get that 4.3% peak flood statistic. There is no automated script inside my project folder to generate this specific shape, so it seems like I just drew it manually on a map back then, and since the paper already says it is an estimate, I just noted it down as a manual guess instead of treating it like a broken bug..... So I also spot-checked three out of the six bibliographies against Google Scholar to make sure they were real, and they looked perfectly fine. (I didn't bother checking the last three references during this round.) Honestly, this was the smoothest verification pass I have ever done on this project, and every single re-run calculation matched the paper perfectly without needing any dirty code fixes.

Entry 12

My research paper's limitations section already points out the biggest problem very clearly. The whole layout depends on just one single treatment area matched against a single control area. Because of this setup, I could never use cluster robust standard errors in my script. Newey West HAC had to do all the heavy lifting to fix the serial correlation issues. (Honestly, it was a pretty risky setup.)

The most obvious way to fix this—which I already wrote down in my Future Work ideas—was to use a proper Synthetic Control Method or jst do a quick sensitivity check using two or three alternative control zones. Decided to run the sensitivity check first. It is the fastest, most direct way to prove if tht -0.0703 drop is a real trend in Kherson or jst a random fluke caused by choosing Tulcea as my only control match.

So I stuck to the exact same logic tht I used to pick Tulcea in the first place. So I did not want to grab some random, super far away control country that would be impossible to defend in my viva. So I picked Galați, Brăila, and Constanța..... These are three Romanian counties touching Tulcea along the same Danube nd Black Sea corridor, meaning they are completely safe from war and share the same mix of floodplains, delta wetlands, and coastal land. Their sizes run from 9,185 to 13,749 km², which sits in the exact same rough ballpark as Tulcea's 16,968 km². Extracted their boundaries directly out of that massive GADM file I already downloaded earlier. (No point wasting internet downloading a fresh file.) I calculated the bounding boxes from those coordinates using the exact same step I used for the Kherson nd Tulcea maps. Then I wrote a script called `download_ndvi_control_zones.py` to pull the exact same NDVI data using the same custom evalscript window nd monthly steps. So I pointed it at these three new boxes n coded three backend scripts to process the raw numbers as soon as they arrived: a pooled multi control DiD script displaying cluster robust and HAC regressions side by side with a per zone chart, a multi control fake placebo test, n a multi control event study graph.

Okayyy so, let's be completely real abt one thing before moving ahead. Having five total data clusters—meaning one treatment zone n four control areas—is definitely a solid upgrade over just having two, which is the absolute bare minimum you need jst to make cluster robust code run without crashing..... But it is still way below the 30 to 40 clusters that heavy econometric theory says you actually need for perfect accuracy. So my plan was to write about the new cluster robust results as a small step in the right direction, not a boastful claim that I built a perfectly balanced multi unit panel dataset. So I chose to keep the HAC errors printed right next to them in the report for tht exact reason.

Entry 13

So yesss, I ran the `download_ndvi_control_zones.py` script. All three new regions downloaded perfectly without any issues..... :) The database got the exact same 35 monthly data points as my Kherson n Tulcea setups, meaning there were zero failed API calls. The raw NDVI numbers came back looking completely normal nd realistic. (Galați and Brăila look slightly greener on average than Tulcea, while Constanța sits somewhere right in the middle.) This minor variation makes sense since they are three distinct geographic landscapes, so it isnt an error or a red flag at all.

The combined pooled regression held up fine. Across all four control areas, the `did_term` value came out to -0.0600, showing an HAC p value of 0.029 nd a cluster robust p value of 0.002. It is a tiny bit smaller than my original score of -0.0703, but it moves in the exact same downward direction n is nowhere near zero. Nxt, I ran a fake placebo test on this combined panel, nd it came back cleanly with a score of +0.0222. (The sign was wrong, nd the p value was high at 0.216, which is exactly wht a clean placebo check should do.)

But when I looked at the individual zone by zone breakdown, things got way more interesting..... Decided not to hide these details inside the combined average number..... Tulcea, Galați, nd Brăila each showed a highly significant impact when tested completely on their own, hitting p values of 0.022, 0.026, nd less than 0.001. But Constanța completely failed to show any real effect, landing at -0.0064 with a totally non significant p value of 0.808. So I started brainstorming why this one region behaves so differently. It is basically the only pure Black Sea beach coast

area, it has way more buildings and cities than the others, and it lacks the muddy river delta floodplain nature that the other three regions share with my main Kherson treatment zone. This feels like a very reasonable guess but I cannot prove it as a fact right now because I haven't run a proper land cover data segmentation test yet. So I just threw this idea into my Future Work list instead of writing it down as a locked in conclusion.

The new cluster robust standard errors came with a big catch. Even on my combined pooled DiD model, the statsmodels package threw a warning saying the cluster covariance matrix was rank deficient. Five clusters is just way too small for heavy asymptotic math rules to work perfectly on their own. (Luckily, the main `did_term` error number still came out looking quite normal and sane.)

But things went completely downhill when I ran the event study graph code. My script was trying to process roughly 24 parameters using only 5 data clusters. Because of this bad ratio, several impact numbers came back with standard errors sitting around $1e-16$. This isn't super high precision at all; it's a genuine math breakdown because there are too many variables trying to squeeze into too few data groups. So I luckily caught this glitch because I actually sat down and read the messy raw printed log terminal instead of blindly trusting the final p values on my screen. So I chose to report the HAC version of the event study instead, and I only included the broken cluster robust numbers to show the reader exactly why I skipped them.

The new event study actually showed a totally different trend compared to my first two zone graph. When I ran the HAC formula on the four control panel, the exact quarter when the dam broke was suddenly not significant anymore with a high p value of 0.972. (In my first draft model, it was highly significant at 0.005.) But the long term impact—measured one full year later—remained highly significant with a clean p value of 0.011. Didn't try to cheat or force these contrasting facts into one perfect, clean storyline. The main, overall pooled model is very stable when you add more control areas, but the detailed quarterly timeline just gets way noisier when you mix data from four totally different natural landscapes. I chose to leave both of these honest facts inside my final research paper exactly as they are.

Sat down and wrote down all these new changes. Added a fresh figure to show the full control panel map, plus an entirely new section in my research paper detailing this multi zone expansion work. So I also updated my project limitations text. The old "one treatment and one control area" issue is now officially upgraded to a "5 cluster problem instead of a solid 30-40 cluster panel" issue. (At least it looks a bit better now.)

Also updated my Future Work list. I removed the old note about control zone sensitivity because that task is finished. In its place, I wrote two new goals: "explain why Constanța behaves differently" and "find a way to scale up to a much larger cluster count." I then made these exact same text updates inside my final Project Report file and the GitHub README page.

Why Constanța doesn't show the exact same environmental drop is still a big mystery to me. I don't want to just guess or make up a reason, so running a proper satellite land cover data check is the only accurate way to test if the beach coastline structure behaves differently than river floodplains. To fix my cluster robust code problem and get a truly reliable cluster count, I will have to find data from outside Romania entirely. I need to look for matching basin areas in Bulgaria or Moldova. Doing that will require massive coordinates searching, boundary downloading, and map matching work which is way too big for this current assignment deadline. So I just threw it onto my long term future task list instead of trying to code it right now.....

Entry 14

The flood extent map was still sitting inside an old QGIS project file, exported through

the QGIS2Web plugin. Wanted to build it the same way I already switched over for my other projects—directly in Python with folium instead of round tripping thru QGIS every single time I need to touch it.

`build_kherson_flood_map.py` reads the exact same three verified UNOSAT shapefiles the static PNG version already uses, June 6, June 9, nd June 21, so nothing new gets pulled in, jst the same trusted data rendered a different way. Kept the exact same colors too—orange for the 6th, red for the 9th, cyan for the 21st—plus the Kherson Oblast boundary outline sitting on top.

Each shapefile turned out to be one big messy MultiPolygon with a ridiculous amount of detail baked in, somewhere between 20K nd 133K vertices depending on the date, so I simplified the geometry down (0.0002° , way under wht you'd even notice at this zoom) before handing it off to folium. Also had to strip the attribute columns first—one of them, `Sensor_Dat`, comes in as a raw pandas Timestamp nd folium's GeoJson serializer jst refuses to JSON encode tht. Wasnt going in the popup anyway since the date nd area were already written straight into the popup HTML from the filename nd `Area_ha`.

Checked the numbers against the static map to make sure nothing got lost in translation: 12,250 ha on June 6, peaking at 46,418 ha on June 9, down to 2,117 ha by the June 21 recession. Matched exactly, so this is jst the same verified data, rendered differently, not a new dataset. Output file came out to abt 2.6MB which GitHub Pages nd the dashboard's iframe embed handle jst fine, same as the other maps.

Updated the dashboard's Interactive Maps page intro nd footer caption, the README's tech stack line nd repo structure comment, nd the Project Report's deliverables line to say Python (folium) instead of QGIS. Left the old `qgis_processing/kherson_flood_extent_webmap/` export sitting there as unused legacy stuff since this app cant delete its own files, it'll stay until I clear it out by hand.

Entry 15

Went back thru `map6_robustness_check.py` nd `map7_control_panel_comparison.py` nd realized both of them had my actual statistical numbers—coefficients, confidence intervals, p values, all of it—jst typed straight into the script as plain Python lists. Tht bugged me once I noticed it, because it means if I ever re run the underlying models nd a number shifts even slightly, these two plots would keep showing the old frozen numbers without me knowing unless I remembered to go back nd retype everything by hand.

So I wrote `generate_model_results.py`, which loads the same NDVI json files nd re fits every single one of my causal models directly—`did_model.py`'s broad baseline, the narrowed baseline, both placebo checks, nd the full multi control panel with the per zone breakdown—nd dumps everything into one `outputs/model_results.json` file. Then I rewired both plotting scripts to jst read their numbers straight out of tht json instead of hvaing anything hardcoded.

Reran both plots after the switch to make sure nothing broke.

`control_panel_comparison.png` came back byte for byte identical. `robustness_check.png` came back visually the same, jst one p value label shows 0.0219 now instead of the rounded 0.0220 I had typed in by hand before, which is jst the script being more precise than I was, not a real change in the result. So now if I ever touch the underlying data again, I jst rerun `generate_model_results.py` first nd both plots update themselves automatically instead of silently going stale.

Entry 16

Okayyy so this one hurts a bit to write down, but tht's literally the whole point of

keeping this log honest, so here goes.

Got an outside review of the project (ran it thru an external AI reviewer jst to get a second pair of eyes on the methodology, not the writing) nd one of the flagged points stopped me cold: it said my NDVI download scripts were querying Sentinel Hub using each zone's bounding box, not its actual GADM polygon shape. My first reaction was honestly kind of defensive, like, I extracted the real boundaries myself back in Entry 3, so surely I used them properly downstream. So I didnt jst take the review's word for it, I went nd actually read `download_ndvi.py` nd `download_ndvi_control_zones.py` line by line myself.

Turns out the review was completely right, nd worse, I'd basically already admitted it to myself nd forgot. `extract_boundaries.py` nd `extract_control_zone_boundaries.py` do pull the real GADM polygon for every zone jst fine, but then both download scripts only ever call `.total_bounds` on tht polygon nd feed the rectangle into the API request, nvr the actual shape. There's even a comment sitting right there in `download_ndvi_control_zones.py` tht literally says "Bounding boxes taken from GADM Level 1," which past me apparently thought was a totally normal thing to write down instead of a red flag. On top of tht, the cloud filtering was only ever scene level (`maxCloudCoverage<=40` before aggregation), no actual pixel level masking of cloud/shadow/cirrus/snow pixels using the Sentinel-2 SCL band. So two real problems, sitting in the pipeline since basically the start.

Decided I wasnt jst going to patch this quietly nd move on. Wrote `export_zone_geometries.py` to simplify each zone's real polygon down to something Sentinel Hub's Statistics API would actually accept in one request (the Danube Delta shapes especially are insanely detailed, Kherson alone came in at 28,406 raw vertices), nd `download_ndvi_polygon.py` to query the true simplified polygon directly instead of the bbox, plus a proper per pixel SCL mask in the evalscript excluding no data, saturated, cloud shadow, cloud, cirrus, nd snow/ice pixels. Output went to a brand new `data/ndvi_v2/` folder so the old bbox based numbers stayed untouched as a baseline to compare against, nothing got overwritten blind.

Ran it. All 5 zones came back with their full 35 monthly points, so no API failures at least. But the comparison against the old data looked genuinely alarming at first glance: valid pixel percentage per zone had collapsed hard, like Kherson's best month capped out around 51% valid instead of nearly 100%, nd the mean NDVI had shifted up almost everywhere by anywhere from 0.05 to 0.15. My first instinct was tht I'd broken something in the evalscript. So I refused to jst accept either explanation (new bug, or new corruption) without checking, nd went nd built a tiny standalone script tht computes each zone's polygon area against its own bounding box area using jst the raw shoelace formula, since I dont even have shapely available in every environment I was checking this from. The area ratios came back almost exactly matching the observed valid% ceiling for every single zone (Kherson ~0.50, Tulcea ~0.66, Galați ~0.66, Constanța ~0.43, Brăila ~0.65). Tht's nt a coincidence, tht's jst wht happens when you clip to the real polygon instead of a rectangle: the pixels sitting inside the bbox but outside the actual shape get correctly marked no data now, when they weren't being excluded before. So this part is a correct fix doing exactly wht it's supposed to do, not a new bug I introduced.

Then came the part I actually didnt want to find. Reran the whole statistical battery — main DiD, both placebos, four county panel, per control breakdown, event study, placebo in space, control only spillover, lag length nd functional form checks, low coverage sensitivity — against the corrected data. The main result moved from -0.0703 (HAC $p=0.022$) to -0.0747 (HAC $p=0.060$). Direction's the same, size is basically the same, but it no longer clears the 0.05 line under its own preferred standard errors. Four county panel did the same thing, -0.0600/ $p=0.029$ to -0.0661/ $p=0.059$.

But the real gut punch was the placebo in space check. Under the old bbox data, Kherson was rank 1 of 5 in the randomization test, the best possible score you can even get with only five units (exact $p=0.20$). Under the corrected data it drops to

rank 2 one sided nd rank 3 two sided (exact $p=0.40$ to 0.60), because Brăila nd Constanța, jst sitting there as ordinary control counties tht nvr had a dam break near them, show effects under the identical fake treatment procedure tht are as big or bigger than Kherson's real one. Ran the control only spillover check again too (Kherson pulled completely out of the panel) nd now two of the four controls move significantly on their own instead of jst one — Constanța was already flagged before, but now Brăila joins it, both showing their own independent shift around the same June 2023 window for reasons I genuinely dont know nd this design cant tell me.

Before jst accepting tht the result is weaker now, I did the one sanity check tht felt like it could let me off the hook: maybe it's jst a handful of extra low coverage months in the new data dragging the number down, since the whole valid% ceiling dropped structurally anyway. Dropped everything below 15% valid, then again below 25% valid, nd reran the main model both times. Didnt help. If anything the p value got slightly worse each time ($0.060 \rightarrow 0.070 \rightarrow 0.083$). So no, it's nt a coverage artifact, the weaker result is jst wht the correctly extracted data actually says.

One small honest silver lining in the middle of all this: the narrowed baseline placebo test, which I'd flagged all the way back in Entry 7 and Entry 10 as a genuine validation failure (fake date inside the narrow window coming back significant under HAC, $p=0.001$), doesnt fail anymore under the corrected data ($p=0.069$ now). So not every single thing got worse, jst most of it, nd I'm writing down both sides rather than only the bad one.

So I went back thru `ECO_Research_Paper.md` top to bottom, Abstract all the way to Conclusion, nd rewrote every section tht touched these numbers to say wht's actually true now instead of wht used to be true, added `ECO_RESULTS_RECONCILIATION.md` to track every single value across every file so nothing gets updated in only one place by accident, nd fixed the README, CITATION.cff abstract, nd the dashboard pages (Vegetation Impact, Statistical Validation, Methodology & Data) to match. Statistical Validation page also got two entirely new sections it never had before, the placebo in space table nd the control only spillover table, since those are honestly the most important findings in this whole entry nd the dashboard had nothing on them at all before this.

Still sitting on my to do list: `data/ndvi_v2/` hasn't been promoted to replace `data/ndvi/` yet, so every other script in this repo (the `map*.py` files, `generate_model_results.py`, etc.) would still silently read the old bbox numbers if I ran them today, nd I want to decide tht properly rather than jst renaming a folder nd hoping nothing downstream breaks. `ECO_Executive_Summary`, this dev log's own PDF export, the Project Report, the EarthArXiv PDF, nd the dashboard's static PDF copies all still show the old, stronger numbers too. Nd I still havent pushed any of this to GitHub main, so right now the published repo is nd the corrected repo on my own machine are telling two different stories, which is nt a place I want to leave this sitting for long. Honestly, this whole entry is exactly why I built all these placebo nd randomization checks into the pipeline in the first place — I jst didn't expect to be the one they caught.

Entry 17

(24 Sept 2026 — a full audit of the repository, done with an AI assistant; this entry records what it found and what changed.)

Entry 16's to-do list is done or superseded: `data/ndvi_v2` had already been promoted to `data/ndvi` (the two folders are byte-identical), but a lot of the repository still showed the old numbers.

What the audit found:

1. **The Phase 2 numbers came from a script that was never in the repo,**

and it used the t distribution while every repo script used the normal distribution, so `did_model.py` printed $p = 0.055$ while the paper said 0.060. The “improvement” in the narrowed placebo ($0.001 \rightarrow 0.069$) was mostly that switch.

2. **The HAC standard errors were wrong for a panel.** Both zones were stacked in one table, so the Newey-West lags ran from the last Kherson month into the first Tulcea month, and same-month observations were treated as independent. All models now run on the single monthly Kherson-minus-control gap (`eco_core.py`), which gives the same coefficients with correctly ordered lags. The primary p-value becomes 0.162.
3. **Seasonality.** Tulcea’s seasonal swing is bigger than Kherson’s and the post period has two summer-autumns against one, so shared month effects leave a seasonal artefact. With zone-specific seasonality the estimate drops to -0.0324 .
4. **Event study:** three pre-event quarters are significant, not one, and every significant post-event quarter is a June–November quarter.
5. **Flood data:** the 6/9 June layers are Sentinel-3 and 21 June is Sentinel-1 (from the shapefiles’ own attributes) — not the five sensors I had been citing. ICEYE on 7 June maps 520.8 km², more than the “9 June peak”; UNOSAT’s cumulative composite is 617 km².
6. **NDVI sampling:** the Statistical API requests never set a pixel size, so every zone was sampled on a 256×256 grid (`sampleCount` 65,536 everywhere), and water was not masked. `download_ndvi_polygon_v3.py` fixes this; I still need to run it with my credentials.
7. **Treatment zone:** the flood covers about 2% of the oblast polygon, and several hundred km² of drained reservoir inside it would raise NDVI, not lower it.

What changed: one engine (`eco_core.py`), one results file (`outputs/model_results.json`), every model script and figure regenerated from it, the dashboard reading its numbers straight from the JSON, flood numbers computed from the shapefiles with sensor labels, pinned requirements, and the paper, README, executive summary and CITATION rewritten to the current numbers. The honest result is now clearer than before: the design does not show a statistically significant vegetation effect of the dam’s destruction.

Entry 18

(24 Sept 2026, later the same day.)

Ran `download_ndvi_polygon_v3.py`. The first run returned only 32 months per zone: the yearly requests ended on 31 December / 30 November, and Sentinel Hub drops a monthly interval that is not complete inside the time range, so Dec 2022, Dec 2023 and Nov 2024 disappeared. With month-boundary end dates the rerun returned all 35 months.

The v3 data (common 0.002° grid, water masked, per-pixel monthly median) change the result again. Primary DiD -0.108 , $p = 0.037$; with zone-specific seasonality -0.069 , $p = 0.005$; the June 2022 placebo is clean; lag lengths, log form and low-coverage checks all hold. So the water pixels and the coarse, zone-dependent sampling had been diluting a real relative decline.

What did not change: Kherson still ranks 2 of 5 in placebo in space — Constanța moves by the same amount — four of five pre-event quarters deviate, and the narrowed placebo fails. The decline is largest in the 2024 season, which looks more like irrigation loss or the war’s effect on farming than like the flood. The honest conclusion is now: a significant, seasonally robust decline in Kherson relative to its controls, which this design cannot attribute to the dam.

`data/ndvi_v3` is now the default dataset (`eco_core.py`); everything was regenerated from it.

